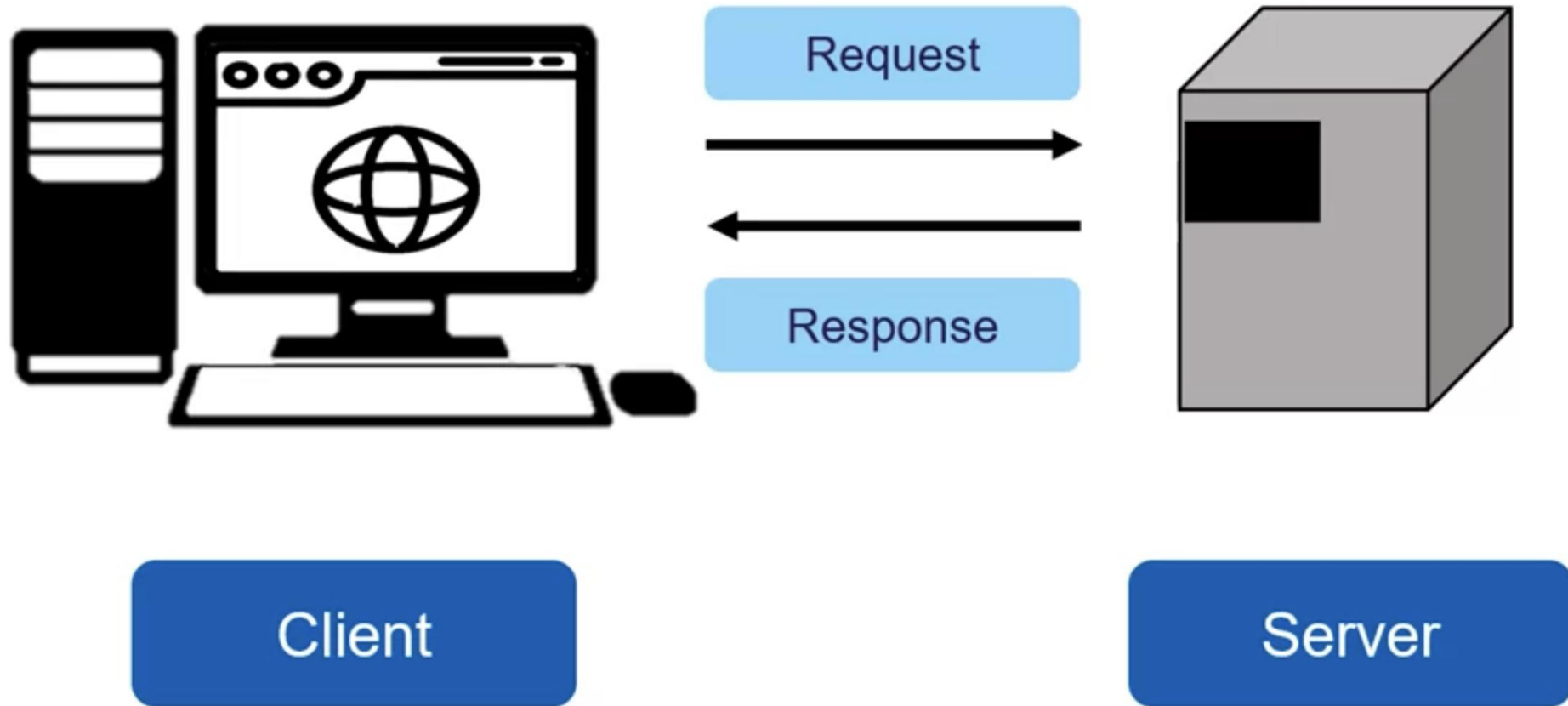


IS311 Web Development

Week 08 - Lecture 09

Dr. Abbas Malik
amaalik@psu.edu.sa

Client Server



Client Side Programming

- Manages behaviours and appearance
- Create interactive web pages
- Communicates with the server
- Uses HTML, CSS and JS



Server Side Programming

- All types of programmes that can run on web server
- Several Languages: Java, Python, C++, PHP, ...
- Delivers customized contents as per users' requests
- Refine responses based on users' data
- JavaScript in Node.js environment





Ryan Dahl in 2009

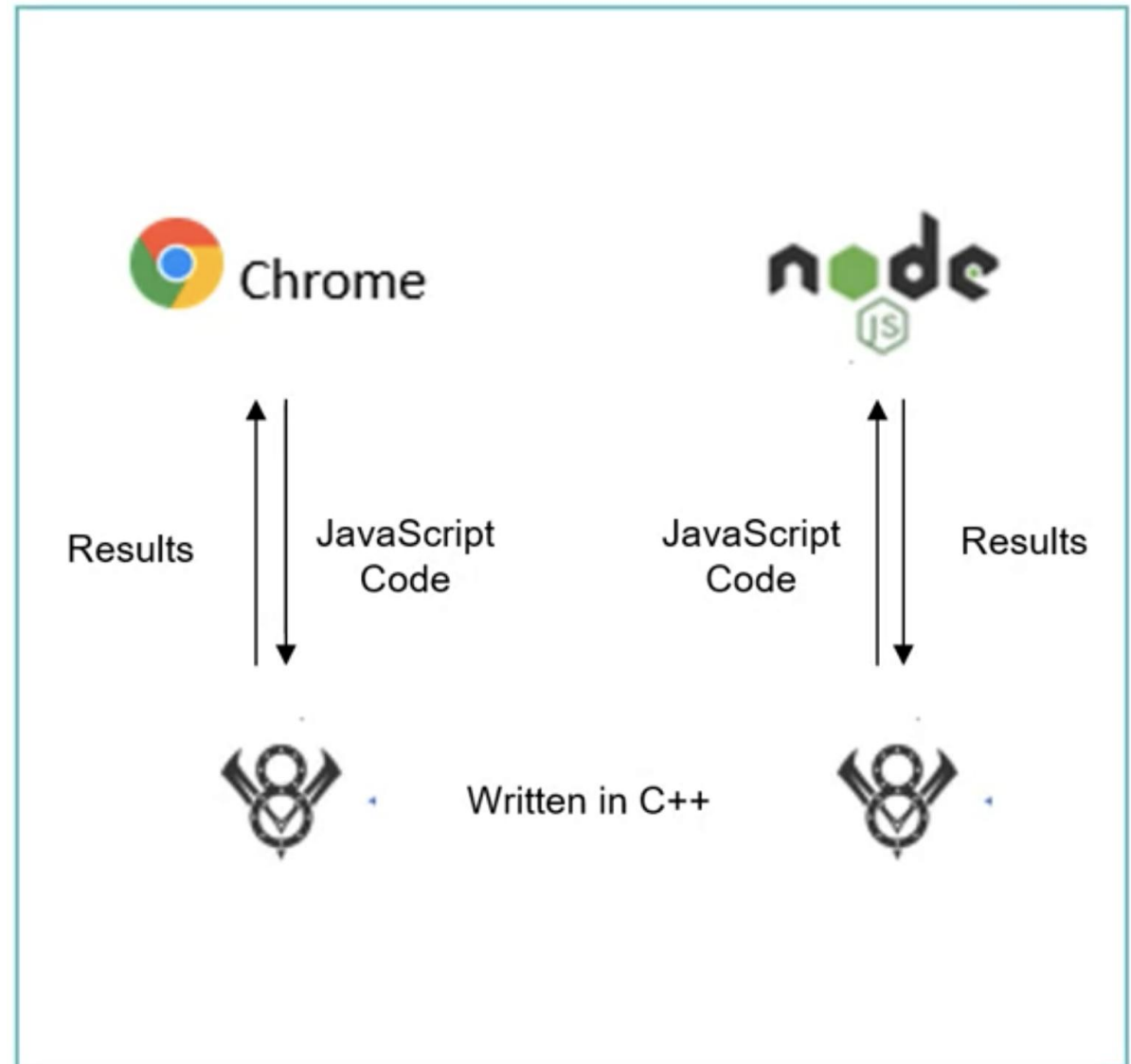
- Open-source server side platform
 - Google Chrome V8 JavaScript engine
- A way to run JavaScript out of browser
- Rich library of modules for web development
- Program in JavaScript



= Runtime Environment + JavaScript Library



- V8 Engine
- Binding through C++
- Browser: works in DOM
- Node.JS: other contexts like network, file system, etc.
- V8 rely on Chrome or Node to provide context



Why

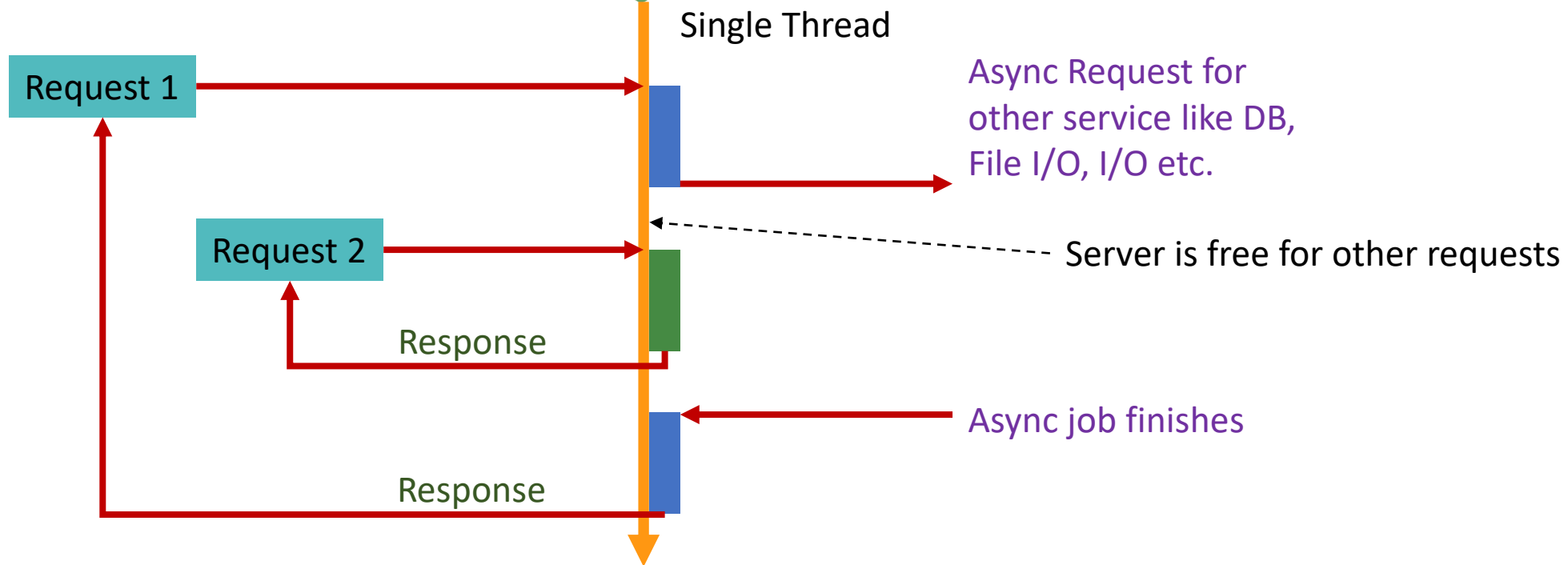
- Efficient to write server-side programs
- Asynchronous and can handle with multiple requests from users
- Efficient users' request processing
- Allow developers to use JavaScript both on Server and on Brower
- Extensive libraries of modules

Features

- All APIs are Asynchronous - non-blocking
- Faster code execution
- Single-thread model
- Do not buffer data
- Notification mechanism of the events - call back functions



Single Thread Model





Helps to build

- Web Servers
 - Real-time servers
 - Socket connections
 - Concurrent connections
- APIs with NoSQL
- CL Utilities

Not for
CPU intensive
apps



Components

Node Command
line interface (CLI)

Node Package
Manager (NPM)

Package.json

Third-party Modules
MySQL, MogoDB, ...

Node Core Modules
http, url, querystring, fs, ...



- Different options to run commands and scripts
- Exposes
 - Built-in debugging
 - Multiple ways to execute scripts
 - Helpful runtime options



NPM

- Useful to install, update, uninstall and configure Node.js modules or packages



package.json

- A plain text file in json format
- Records important metadata of project
- Define info like project name, module dependencies, module version, etc.

Build first Node.JS program

- Create a first.js file in a folder on your computer
- Write the following code in this file

```
console.log("First test of Node JS");
```

- Open terminal or command prompt in this folder and run command

```
node first.js
```

Build first Node.JS program

The screenshot shows the Visual Studio Code interface. On the left, the 'EXPLORER' sidebar shows a file named 'first.js' under the 'CODE' section. The main editor area displays the content of 'first.js', which is a single line of JavaScript code: `console.log("First test of Node JS");`. Below the editor, the 'TERMINAL' tab is active, showing the command `node first.js` being executed. The output of the command is `First test of Node JS`.

```

EXPLORER
  CODE
    first.js

JS first.js
1 console.log("First test of Node JS");

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• m.g.abbasmalik@Ms-MacBook-Pro Code % node first.js
  First test of Node JS
○ m.g.abbasmalik@Ms-MacBook-Pro Code %
  
```



Core Modules

- Reusable modules
- Core Modules: present in the NODE.JS
 - Helps developers to build their own modules
- Can import a module in the new module or app

```
require('http');
```

Global

- In Browser: **Window** is global scope or context
- In Node: **global** is the global scope of context

```
// firstNode.js file  
console.log(global);
```

Global

```
<ref *1> Object [global] {
  global: [Circular *1],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function: setImmediate] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function: setTimeout] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  queueMicrotask: [Function: queueMicrotask],
  structuredClone: [Function: structuredClone],
  atob: [Getter/Setter],
  btoa: [Getter/Setter],
  performance: [Getter/Setter],
  navigator: [Getter],
  fetch: [Function: fetch],
  crypto: [Getter]
}
```

Global

- In browser, if we define a variable or function, it becomes part of the global scope (window)
- But in Node, it does not become part of **global**
- Insead, they exists only in the file/module

```
var x = 4.56;
console.log(global.x);
```

```
m.g.abbasmalik@Ms-MacBook-Pro Code % node firstNode.js
undefined
```

[global docs](#)

Module

- A JS file
- All variables and functions defined: local or private to this file/module
- Let us log module and see what it contains.

Module

```
console.log(module);
```

```
• m.g.abbasmalik@Ms-MacBook-Pro Code % node firstNode.js
{
  id: '.',
  path: '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code',
  exports: {},
  filename: '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code/firstNode.js',
  loaded: false,
  children: [],
  paths: [
    '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code/node_modules',
    '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/node_modules',
    '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/node_modules',
    '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/node_modules',
    '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/node_modules',
    '/Users/m.g.abbasmalik/Teaching/PSU/Courses/node_modules',
    '/Users/m.g.abbasmalik/Teaching/PSU/node_modules',
    '/Users/m.g.abbasmalik/Teaching/node_modules',
    '/Users/m.g.abbasmalik/node_modules',
    '/Users/node_modules',
    '/node_modules'
  ]
}
```

Build first module

- Create a "firstmodule" folder.
- Create a new file multiplicationtable.js
- Create another new file app.js
- Edit the multiplicationtable.js
 - Add two functions in this file
 - Export or expose one function out of this module

multiplicationtable.js

```
// building a module to print a multiplication table
// this is our first attempt to make a module
```

```
function printTable(num){
  for(let i = 1; i <= 10; i++){
    printLine(i, num);
  }
}
```

```
function printLine(i, num){
  console.log(num + " * " + i + " = " + (i*num));
}
```

```
module.exports.printTable = printTable;
```

app.js

```
// importing the module as an object  
var mt = require('./multiplicationtable.js')  
// use module to call exposed function  
mt.printTable(3);
```

```
m.g.abbasmalik@Ms-MacBook-Pro firstmodule % node app.js  
3 * 1 = 3  
3 * 2 = 6  
3 * 3 = 9  
3 * 4 = 12  
3 * 5 = 15  
3 * 6 = 18  
3 * 7 = 21  
3 * 8 = 24  
3 * 9 = 27  
3 * 10 = 30
```

app.js

printLine function is private to module

```
var mt = require('./multiplicationtable.js')
console.log(mt);
mt.printTable(3);
```

```
m.g.abbasmalik@Ms-MacBook-Pro firstmodule % node app.js
{ printTable: [Function: printTable] }
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Second module

- Let us add another file logger.js

```
function logger(message){  
    console.log(message);  
}  
  
module.exports.logger = logger;
```

app.js

```
var mt = require('./multiplicationtable.js')
var logger = require('./logger.js')
mt.printTable(3);
logger.logger("Hello Node World!");
```

```
m.g.abbasmalik@Ms-MacBook-Pro firstmodule % node app.js
{ printTable: [Function: printTable] }
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
Hello Node World!
```

module

- Best practice to define imported module as **const**

```
const mt = require('./multiplicationtable.js');
const logger = require('./logger.js');
```

```
var mt = require('./multiplicationtable.js');
var logger = require('./logger.js');
mt = false;
mt.printTable(3);
```

module

- Other important local properties
- `__filename`
- `__dirname`

```
console.log("FileName: ", __filename);
console.log("Directory: ", __dirname);
```

```
FileName: /Users/m.g.abbasalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code/firstmodule/a
pp.js
Directory: /Users/m.g.abbasalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code/firstmodule
```



Builtin Modules

<https://nodejs.org/docs/latest/api/>

- Path
- OS
- HTTP
- HTTPS
- Events
- Filesystem

Path Module

<https://nodejs.org/docs/latest/api/path.html>

JS app_path.js X

JS app_path.js > ...

```
1 const path = require('path');
2 var pathObj = path.parse(__filename);
3 console.log(pathObj);
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

• m.g.abbasalik@Ms-MacBook-Pro Code % node app_path.js

```
{
  root: '/',
  dir: '/Users/m.g.abbasalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code',
  base: 'app_path.js',
  ext: '.js',
  name: 'app_path'
}
```

◦ m.g.abbasalik@Ms-MacBook-Pro Code %

OS Module

<https://nodejs.org/docs/latest/api/os.html>

```
const os = require('os');
var totalMemory = os.totalmem();
var freeMemory = os.freemem();
console.log("Total Memory: " + totalMemory);
console.log(`Total Memory: ${totalMemory}`);
console.log(`Free Memory: ${freeMemory}`);
```

```
m.g.abbasmalik@Ms-MacBook-Pro Code % node app_os.js
Total Memory: 34359738368
Total Memory: 34359738368
Free Memory: 334049280
```

Filesystem Module

<https://nodejs.org/docs/latest/api/fs.html>

The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'firstmodule' directory and several JavaScript files. The code editor shows the content of 'app_fs.js', which uses the 'fs' module to read the contents of the current directory synchronously. The terminal at the bottom shows the command 'node app_fs.js' being executed, resulting in an array of file names in the current directory.

```

EXPLORER
  CODE
    firstmodule
    JS app_fs.js
    JS app_os.js
    JS app_path.js
    JS firstNode.js

JS app_fs.js
1 const fs = require('fs');
2
3 const files = fs.readdirSync('./');
4
5 console.log(files);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• m.g.abbasmalik@Ms-MacBook-Pro Code % node app_fs.js
[
  '.DS_Store',
  'app_fs.js',
  'app_os.js',
  'app_path.js',
  'firstNode.js',
  'firstmodule'
]
  
```

Synchronous Call
Blocking Thread

Filesystem Module

```
const fs = require('fs');

fs.readdir('./', function(err, res) {
  if (err)
    console.log(err);
  Else
    console.log("Results:", res);
});
```

```
m.g.abbasmalik@Ms-MacBook-Pro Code % node app_fs_async.js
Results: [
  '.DS_Store',
  'app_fs.js',
  'app_fs_async.js',
  'app_os.js',
  'app_path.js',
  'firstNode.js',
  'firstmodule'
]
```

Process Module

<https://nodejs.org/docs/latest/api/process.html>

- Modify the app.js in firstmodule folder

```
const pr = require('process');
const mt = require('./multiplicationtable.js');
const logger = require('./logger.js');
// getting command line arguments
var args = pr.argv
console.log("arguments: ", args);
mt.printTable(args[3]);
logger.logger(args[2]);
```

Process Module

```
m.g.abbasmalik@Ms-MacBook-Pro firstmodule % node app.js 'message from cl' 5
arguments: [
  '/usr/local/bin/node',
  '/Users/m.g.abbasmalik/Teaching/PSU/Courses/IS311-Web Development/IS311 Web Dev Course/Lectures/Week 08/Code/firstmo
dule/app.js',
  'message from cl',
  '5'
]
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
message from cl
```

```
const pr = require('process');
const mt = require('./multiplicationtable.js');
const logger = require('./logger.js');
// getting command line arguments
var args = pr.argv
console.log("arguments: ", args);
mt.printTable(args[3]);
logger.logger(args[2]);
```

Events

<https://nodejs.org/docs/latest/api/events.html>

- **EventEmitter** - a class to emit and handle events

```
// this is class EventEmitter
const EventEmitter = require('events');
// create class object to use it
const emitter = new EventEmitter();
// create listner for the event to respond to the event when it happens
emitter.on('myEvent', function (){
    console.log('myEvent: custom event is raised and handled');
});
// raised an event
emitter.emit('myEvent');
```

Events

- In real world, **EventEmitter** class is used as parent to add event handling capabilities in a real world
- Let us build an adder module that can add two numbers and save results in a sum variable in the module
- This module will raise the event that numbers are added that will handle in the app.js to show the results.

Adder

- Start a new Project in a new Folder
- Create adder.js file and edit it.

```
var sum;  
function add(num1, num2){  
    num1 = num1 || 0;  
    num2 = num2 || 0;  
    sum = num1 + num2;  
}  
  
exports.add = add;  
exports.result = sum;
```

App

- Create app.js and edit it

```
const adder = require('./adder.js');  
adder.add(34, 56);
```

Adder

- Now we want that in the adder, we raise an event that numbers are added when two numbers are added. So modify the adder.js
- Import EventEmitter, create its object and emit the event in the add function

Adder

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
var sum;
function add(num1, num2){
  num1 = num1 || 0;
  num2 = num2 || 0;
  sum = num1 + num2;
  emitter.emit('addedEvent');
  console.log('numbers are added and event raised!!!');
}
exports.add = add;
exports.result = sum;
```

App

- We want to handle 'addedEvent' in app to show results on the console. So edit app.js
- Import EventEmitter and create its object.
- Add an event listener to it and print addition results in console

App

```
const adder = require('./adder.js');
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('addedEvent', function(){
  console.log('Sum of numbers is ', adder.result);
});
adder.add(34, 56);
```

```
m.g.abbasmalik@Ms-MacBook-Pro events % node app.js
numbers are added and event raised!!!
```

Why Event Listener is not working?

```
app.js > ...
const adder = require('./adder.js');
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('addedEvent', function(){
  console.log('Sum of numbers is ', adder.result);
});
adder.add(34, 56);
```

```
events > JS adder.js > add
1 const EventEmitter = require('events');
2 const emitter = new EventEmitter();
3 var sum;
4 function add(num1, num2){
5   num1 = num1 || 0;
6   num2 = num2 || 0;
7   sum = num1 + num2;
8   emitter.emit('addedEvent');
9   console.log('numbers are added and event raised!!!');
10 }
11 exports.add = add;
12 exports.result = sum;
```

Solution: Class and Inheritance

Modified adder.js

```
const EventEmitter = require('events');  
class Adder extends EventEmitter{  
  sum;  
  add(num1, num2){  
    num1 = num1 || 0;  
    num2 = num2 || 0;  
    this.sum = num1 + num2;  
    this.emit('addedEvent');  
  }  
};  
module.exports = Adder;
```

Solution: Class and Inheritance

Modified app.js

```
const Adder = require('./adder.js');
const adder = new Adder();
adder.on('addedEvent', function(){
  console.log('Sum of numbers is', adder.sum);
});
adder.add(34, 56);
```

```
m.g.abbasmalik@Ms-MacBook-Pro events % node app.js
Sum of numbers is 90
```

HTTP Module

<https://nodejs.org/docs/latest/api/http.html>

- Network Application
- Web Server
- Let us build a web server than receive requests and respond to them

Web Server

- Create a new folder http
- Create a app.js file and edit it

App

```
const http = require('http');

const serverObj = http.createServer(); // an EventEmitter
serverObj.listen(3000); // starting to listen on port #
// Registering a listener
serverObj.on('connection', (socket)=>{
  console.log('new connection is made!!');
});

console.log('Listening on port 3000...');
```

First WebServer for HTTP Requests

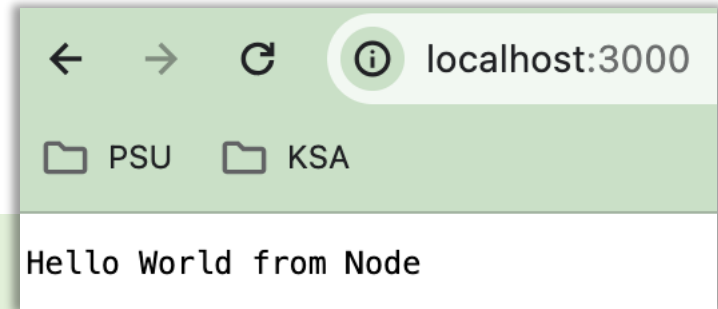
- Create a new file firstserver.js and edit
- Import HTTP
- Create Server and pass a call back function that can receive requests and send back response to browser based on URL

firstserver.js

```
const http = require('http');

const serverObj = http.createServer(
  (request, response) => { // callback function
    if(request.url === '/') {
      response.write('Hello World from Node');
      response.end();
    }
  }
);

serverObj.listen(3000);
console.log('Listening on port 3000...');
```



```
m.g.abbasmalik@Ms-MacBook-Pro http % node firstserver.js
Listening on port 3000...
```

Add route for adder module

http > JS firstserver.js > ...

```
1  const http = require('http');
2  const Adder = require('../events/adder');
3  const adder = new Adder();
4
5  const serverObj = http.createServer(
6    (request, response) => { // callback function
7      if(request.url === '/'){
8        response.write('Hello World from Node');
9        response.end();
10     }
11     if(request.url === '/add'){
12       adder.on('addedEvent', function(){
13         response.write(`Sum of numbers is ${adder.sum}`);
14         response.end();
15       });
16       adder.add(34, 56);
17     }
18   }
19 );
20 serverObj.listen(3000);
21 console.log('Listening on port 3000...');
```

← → ↻ ⓘ localhost:3000/add

📁 PSU 📁 KSA

Sum of numbers is 90

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
m.g.abbasmalik@Ms-MacBook-Pro http % node firstserver.js
Listening on port 3000...
```

References

- <https://www.w3schools.com/nodejs/default.asp>
- <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>